

PROGRAMMATION FONCTIONNELLE 2 : TD3

janvier 2026 — Pierre Rousselin

Exercice 1 : Incrémenter, décrémenter

On considère la fonction OCaml

```
let foo = fun x -> fun y -> if x then y + 1 else y - 1
```

1. Quel est le type de `foo` ?
2. Que vaut `foo false` ? Quel est son type ?
3. Que vaut `foo true 41` ? Quel est son type ?
4. Que vaut `fun b -> foo b 42` ? Quel est son type ?
5. Que vaut `fun b -> foo b 0 = 1` ? Quel est son type ?
6. Écrire une définition équivalent de `foo` utilisant :
 - a) un seul `fun`
 - b) aucun `fun` (contrairement à cet exercice, huhuhu).

--- * ---

Correction de l'exercice 1 :

1. `foo : bool -> int -> int`
2. `foo false` se réduit en `fun y -> if false then y + 1 else y - 1` donc sera (extensionnellement) égal à `fun y -> y - 1`. Son type est `int -> int`
3. `foo true 41` se réduit tout d'abord en `((fun y -> if true then y + 1 else y - 1) 41)` puis en `if true then 41 + 1 else 41 - 1` dont la valeur est 42.
4. `fun b -> foo b 42` est de type `bool -> int`. Si `b` vaut `true`, l'évaluation donne 43, sinon elle donne 41.
5. `fun b -> foo b 0 = 1` est de type `bool -> bool`. Si son argument vaut `true`, elle vaut `true`, sinon elle vaut `false`. Bref, c'est une façon compliquée de calculer l'identité sur `bool`.
6. a) `let foo = fun x y -> if x then y + 1 else y - 1`
 b) `let foo x y = if x then y + 1 else y - 1`

--- * ---

Exercice 2 : Curryfier, décurryfier

On considère la fonction OCaml

```
let curry = fun f -> fun x -> fun y -> f (x, y)
```

1. Donner le type de la fonction `curry`. On pourra utiliser les variables de type `'a`, `'b`, ...
2. Définir la fonction réciproque `uncurry` et donner son type.
3. Donner plusieurs définitions équivalentes à `uncurry foo`, sans utiliser `uncurry`, où `foo` est la fonction de l'exercice précédent.

--- * ---

Correction de l'exercice 2 :

1. Si `x` est de type `'a` et `y` de type `'b`, on voit que `f` doit être une fonction dont l'argument est un couple de type `'a * 'b`, elle retourne un type quelconque `'c`. Tout mis ensemble, on a
`curry : ('a * 'b -> 'c) -> 'a -> 'b -> 'c`
2. `let uncurry f = fun (x, y) -> f x y`

ou toute autre définition équivalente

3.

```
let foo_uncurried c = match c with (x, y) -> if x then y + 1 else y - 1
let foo_uncurried c = let (x, y) = c in if x then y + 1 else y - 1
let foo_uncurried (x, y) = if x then y + 1 else y - 1
```

--- * ---

Exercice 3 : Terme de preuve

Donner 2 preuves de chacun des lemmes suivants : une preuve avec juste `exact` et une preuve utilisant `apply`, `destruct`, `split`, etc.

- `Lemma td3ex3a (A B C : Prop) : A -> B -> (A -> B -> C) -> C.`
- `Lemma td3ex3b (A B C : Prop) : (A -> B -> C) -> (A /\ B -> C).`
- `Lemma td3ex3c (A B C : Prop) : (A /\ B -> C) -> (A -> B -> C).`

Que dire des preuves de `td3ex3b` et `td3ex3c` ?

--- * ---

Exercice 4 : Dédution naturelle

En fonction des besoins et du temps restant dans le TD, vous pouvez prouver les lemmes de l'exercice précédent avec des arbres de preuves en déduction naturelle.

--- * ---

Correction de l'exercice 4 :

`Lemma td3ex3a (A B C : Prop) : A -> B -> (A -> B -> C) -> C.`

`Proof.`

```
intros hA hB hABC.
apply hABC.
- exact hA.
- exact hB.
```

`Qed.`

`Lemma td3ex3a' (A B C : Prop) : A -> B -> (A -> B -> C) -> C.`

`Proof.`

```
exact (fun hA hB hABC => hABC hA hB).
```

`Qed.`

`Lemma td3ex3b (A B C : Prop) : (A -> B -> C) -> (A /\ B -> C).`

`Proof.`

```
(* intros h [? ?]; apply h; assumption. *)
intros hABC hAB.
destruct hAB as [hA hB].
apply hABC.
- exact hA.
- exact hB.
```

`Qed.`

`Lemma td3ex3b' (A B C : Prop) : (A -> B -> C) -> (A /\ B -> C).`

`Proof.`

```
(* C'est uncurry dans Prop. *)
exact (fun f '(conj hA hB) => f hA hB).
```

`Qed.`

`Lemma td3ex3c (A B C : Prop) : (A /\ B -> C) -> (A -> B -> C).`

`Proof.`

```
(* intros h ? ?; apply h; split; assumption. *)
```

```
intros h hA hB.
apply h.
split.
- exact hA.
- exact hB.
Qed.

Lemma td3ex3c' (A B C : Prop) : (A /\ B -> C) -> (A -> B -> C).
Proof.
  (* C'est curry dans Prop. *)
  exact (fun f hA hB => f (conj hA hB)).
Qed.
```

--- * ---